A3-PS022-TC005

Segfault: Drop the Flood, Keep the Queries

PS-022: eBPF/XDP Rate Limiting for DNS Query Floods in a Lab

Debangan Mali · **Saraswata Chatterjee** · **Partha Chakraborty** (Faculty)
debanganmali99@gmail.com

Team Segfault, Institute of Engineering and Management, Kolkata

AIORI Hackathon 3

October 11, 2026

Table of Contents

1. Problem Description 3
2. Background 4
3. Literature and RFC Review 5
4. Proposed Solution 6
5. Solution Optimization 9
6. Solution Architecture and Design 12
7. Data Flow Diagram 13
8. Evaluation Plan 14
9. Real-World Deployment 15
10. Timeline of Delivery 16
11. Conclusion 17
12. References 18

Numbers on the right are slide numbers. All performance figures in this deck are targets, to be validated, unless marked measured.

Problem Description

Under a DNS query flood, a Linux server fails twice: it burns CPU on every packet, and then drops packets blindly.

- **Cost:** each packet pays about 1 to 3 µs of kernel work (skb allocation, netfilter, socket) before any drop.
- **Blind drops:** a full socket queue drops legitimate and attack queries alike.
- **Spoofing:** forged source addresses defeat per-source limits.
- **Pulses, random subdomains:** bursts beat slow averages; unique names bypass the resolver cache.

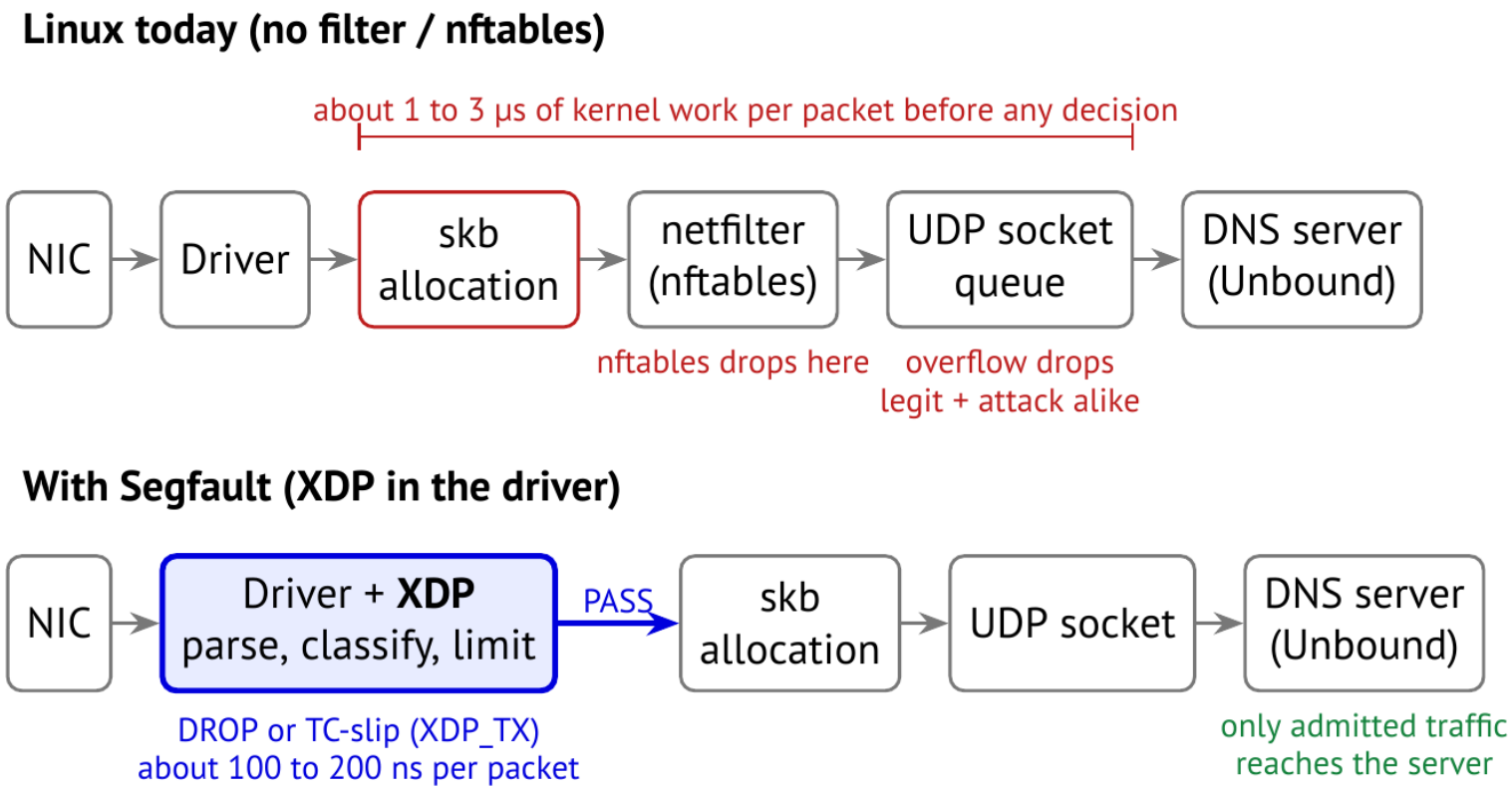
PS-022 asks for

- one Linux host pair in a lab
- bounded UDP/DNS parsing
- per-source and global limits
- 3 synthetic flood patterns
- comparison with no filter and nftables
- throughput, CPU, legit success, drops

Out of scope: public floods, full DNS semantics, universal PRSD detection, eliminating reflection.

Background

XDP runs eBPF code inside the NIC driver, before the kernel allocates a socket buffer [1].



A spoofer never receives replies, so it cannot complete a TCP handshake [8] or echo a server cookie [9, 10]: both prove a source is real.

Literature & RFC Review

XDP rate limiting for DNS already exists; Segfault builds on it and fixes where it breaks.

Prior work	What it does	Where it falls short	What Segfault adds
NLnet Labs XDP RRL [2, 3]	Per-source rate limiting in XDP	State churned by spoofed floods	Sketch-first state, lockless GCRA [5, 17]
NLnet Labs XDP Cookies [4]	Cookie holders skip limits	Only cookie-capable clients; no global fairness	TCP verifier (sockops); leased global pool
BIND / Unbound RRL [16]	Limits inside the DNS server	Full kernel cost already paid	Decision in the driver
nftables meter + limit	Limits in netfilter	After skb allocation; no verification	Earlier, cheaper, verification-aware
Sketches in eBPF [6]	In-kernel measurement	Measurement only	Drives promotion to exact limiters

Standards used: RFC 1035 [7], RFC 7766 DNS over TCP [8], RFC 7873 and RFC 9018 DNS Cookies [9, 10], RFC 8198 aggressive NSEC [11], RFC 8955 FlowSpec [12], RFC 9132 DOTS [13].

Proposed Solution

One XDP program with three cheap layers, run cheapest first, plus slow-path kernel hooks for proof and feedback.

1. **Bounded parse and sanity:** 2 VLANs, IPv4/6 (up to 4 extension headers), QNAME up to 255 B; fragments and malformed headers dropped.

2. **Spoof-resistant state:** Count-Min Sketch first; only heavy hitters get an exact GCRA limiter.

3. **Verified priority:** clients proven by a DNS cookie (checked in XDP [14]) or a TCP handshake (sockops) skip the global and zone limits.

Over budget: TC-slip (truncated reply, rate-limited) so real resolvers retry over TCP; otherwise drop.

Class	Treatment under attack
Verified	Cookie or TCP proof; skips zone and pool limits; still per-source limited
Familiar	Seen often in calm periods; reserved share of the pool
Unknown	Shares the rest; over budget is slipped or dropped

Feedback: TC egress reads response codes per zone; a 50 ms kernel timer runs the CALM, SUSPECT, ATTACK modes.

Proposed Solution: the Three Scenarios

Each flood pattern is beaten by a different layer, and each layer stays cheap under the attack it targets.

Scenario	How Segfault resolves it	Attack-path cost
A Single-source flood	Sketch flags the source within a few packets; an exact GCRA entry drops the excess; repeat offenders get short bans.	about 90 ns/pkt
B Distributed spoofed flood	Random sources are never promoted (no churn); prefix keys (/24 to /16, /64 to /48) catch clusters; the leased pool caps unknown traffic near 90% of server capacity; verified clients keep their share.	about 170 ns/pkt
C Pulsed random-subdomain flood	GCRA acts per packet, so pulses get no free window; fast-attack, slow-decay modes; zone sketch plus NXDOMAIN ratio flags the hammered zone; a good-name cache keeps popular names working.	about 200 ns/pkt
D Flash crowd (extra test)	High familiar share and low NXDOMAIN keep the mode in SUSPECT; no slips to familiar clients.	false-positive test

Scenario C is bounded mitigation of a hammered zone, not universal PRSD detection. Costs are estimates to be measured.

Proposed Solution: Per-Packet Pseudocode

```
xdp_main(pkt):
  p = parse_bounded(pkt)             # <=2 VLAN, IPv4/6 + <=4 ext hdrs, UDP/TCP, QNAME <=255B
  if !p.dns:            return PASS
  if p.frag_to_dns:     return frag_policy(p)              # drop or per-source GCRA
  if !hdr_sane(p):      return DROP                        # QR=0, opcode, section counts
  if p.tcp:             return syn_guard(p)                # per-source SYN GCRA
  key = p.cookie_ok ? (p.src, p.client_cookie) : p.src     # CGNAT users counted apart
  cls = p.cookie_ok or verified[p.src] ? VERIFIED : familiar[p.src] ? FAMILIAR : UNKNOWN
  e = src_table.lookup(key)                                # lookup only
  if !e:
     est = cms_add(key, prefixes(p.src), epoch(now))       # conservative update, epoch tags
     if est > promote_th(mode, n_epoch) and gcra(&insert_rate): e = src_table.insert(key)
  if e and !gcra(&e.tat, rate[cls]):       return deny(p, e)
  if cls == VERIFIED:                      return PASS
  if mode != CALM and zone_flagged(p.zone) and !good_name[p.qname]:
     if !gcra(&zone[p.zone].tat, zone_rate):  return deny(p, e)
  if !lease_token(cls, mode):              return deny(p, e)   # ceiling always on
  return PASS

deny(p, e):
  strike(e)                                                # bans UNKNOWN only, short
  if slip_ok(p) and !tcp_blocked(prefix(p.src)):
     build_tc_reply(p, server_cookie_if_client_sent_one); return XDP_TX
  return DROP
```

Optimization Proposed

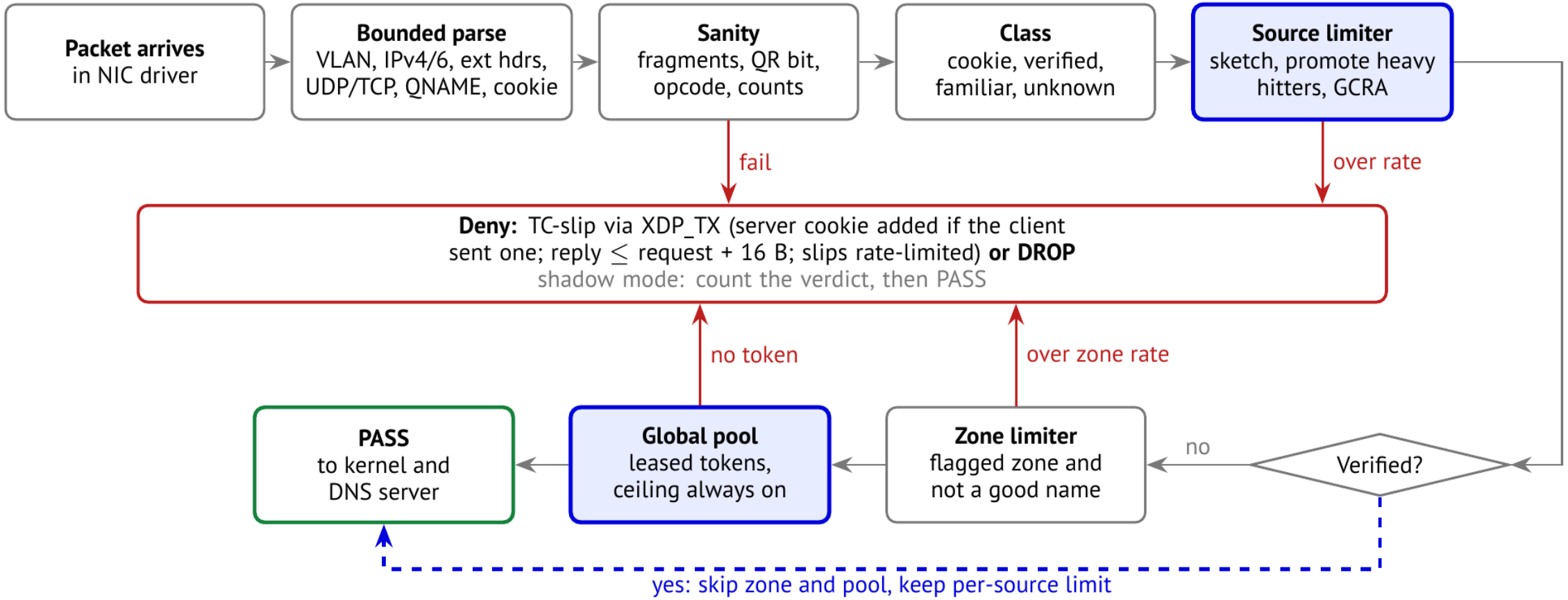
Eight engineering choices turn a basic XDP limiter into one that stays fast and fair under spoofing.

Concern	Before (basic XDP limiter)	After (Segfault)
State under spoofing	LRU insert per new source; legit buckets evicted	Sketch first; only heavy hitters inserted; inserts rate-capped
Per-source limiter	Token bucket with a lock	GCRA: one 64-bit timestamp, one cmpxchg [17]
CPU contention	Shared LRU free list	Per-CPU LRU lists; small promoted set
Global budget	Fixed per-CPU split, unfair under RSS skew	Shared pool leased 64 tokens per atomic; always on
Reaction to pulses	Userspace polling	Kernel timer plus lazy tick in the datapath, ≤ 100 ms
Slip safety	Reply can act as a reflector	Reply ≤ request (+16 B with cookie); slips rate-limited
Real-user fairness	One IP = one user	CGNAT users keyed by client cookie; verified share
Config cost	Map lookup per packet	mmapped config, zero lookups, atomic hot-reload

GCRA: emission interval $T = 1/r$, tolerance $\tau = (B - 1)\cdot T$. Deny if $t < TAT - \tau$, else $TAT \leftarrow \max(TAT, t) + T$.

Optimization Process

Every packet takes the cheapest path that can decide it; only one stage touches shared state.



Optimization: Targets

Definition of done at about 10x server capacity (targets, to be validated).

Scenario	Legit success	p99 latency	Reaction
A single source	≥ 99.5%	< 2x baseline	per packet
B spoofed, distributed	≥ 98%	< 3x baseline	≤ 100 ms
C pulsed random-subdomain	≥ 98%	< 3x baseline	≤ 100 ms

Guarantees hold while the network link itself is not saturated; link-filling floods are handed upstream (FlowSpec, DOTS).

Path	Estimate
Calm legit query	150 ns
Spoofed packet, drop	170 ns
Promoted attacker, drop	90 ns
Slip reply (XDP_TX)	250 ns
Kernel path avoided	**1 to 3 µs**

Measured later with kernel BPF statistics (ns per packet).

Solution Architecture & Design

Only the XDP program touches every packet; the other components run off the hot path.

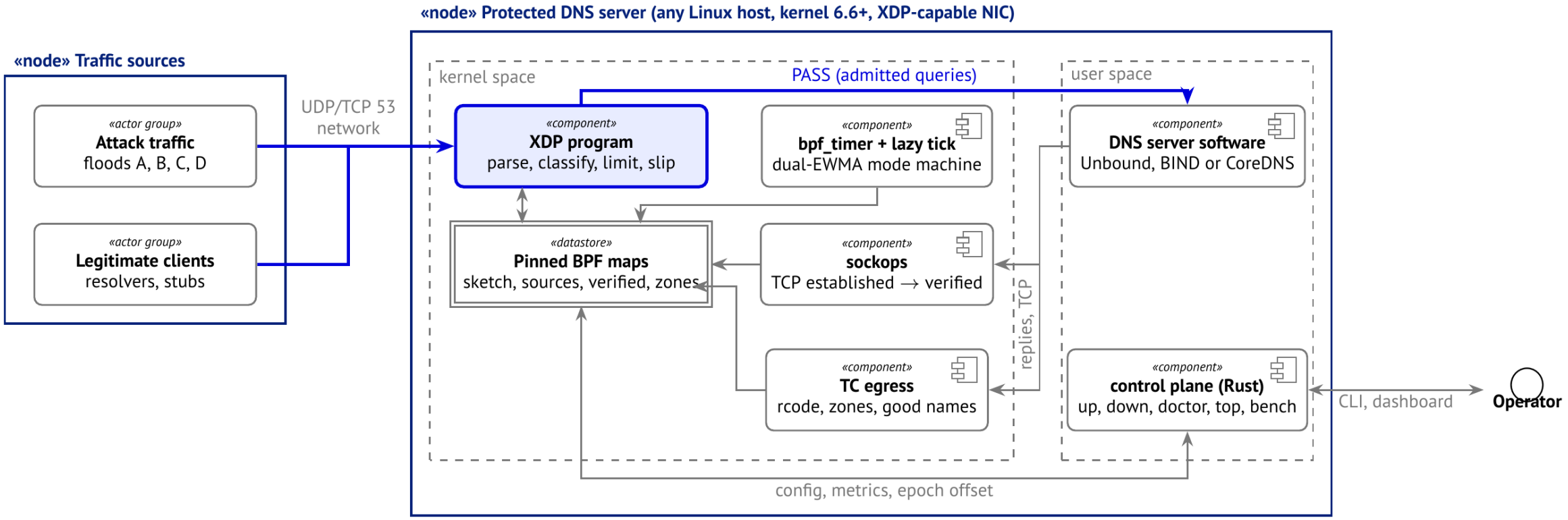


Figure: System Architecture UML Diagram (deployment and components)

Data Flow Diagram

Queries flow left to right; learning and control loops feed the stores the datapath reads.

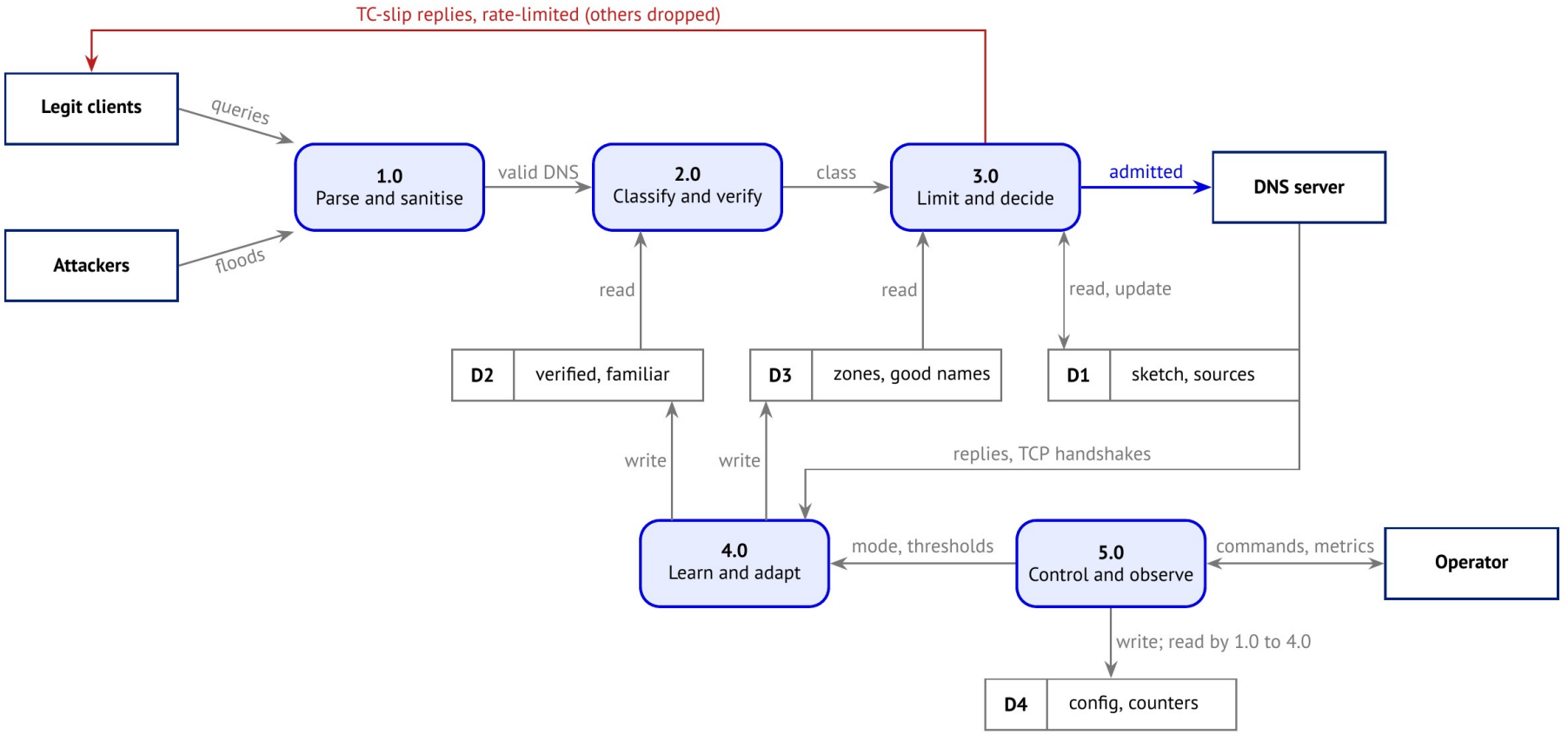


Figure: Data Flow Diagram (DFD), level 1

Evaluation Plan and PS-022 Compliance

One command, segfault bench run, produces every number.

- 4 scenarios x 5 modes x 5 repeats: no filter, nftables, basic XDP, Segfault, Segfault generic mode
- Ablation: remove one feature at a time to show its value
- Open-loop legit client (no hidden latency), cookie and TCP aware
- Metrics: pps, CPU, softirq, steal time, legit success, p50/p99, drops by stage, ns/pkt
- 10 s warmup, 60 s run, 95% confidence intervals; offered load verified per run

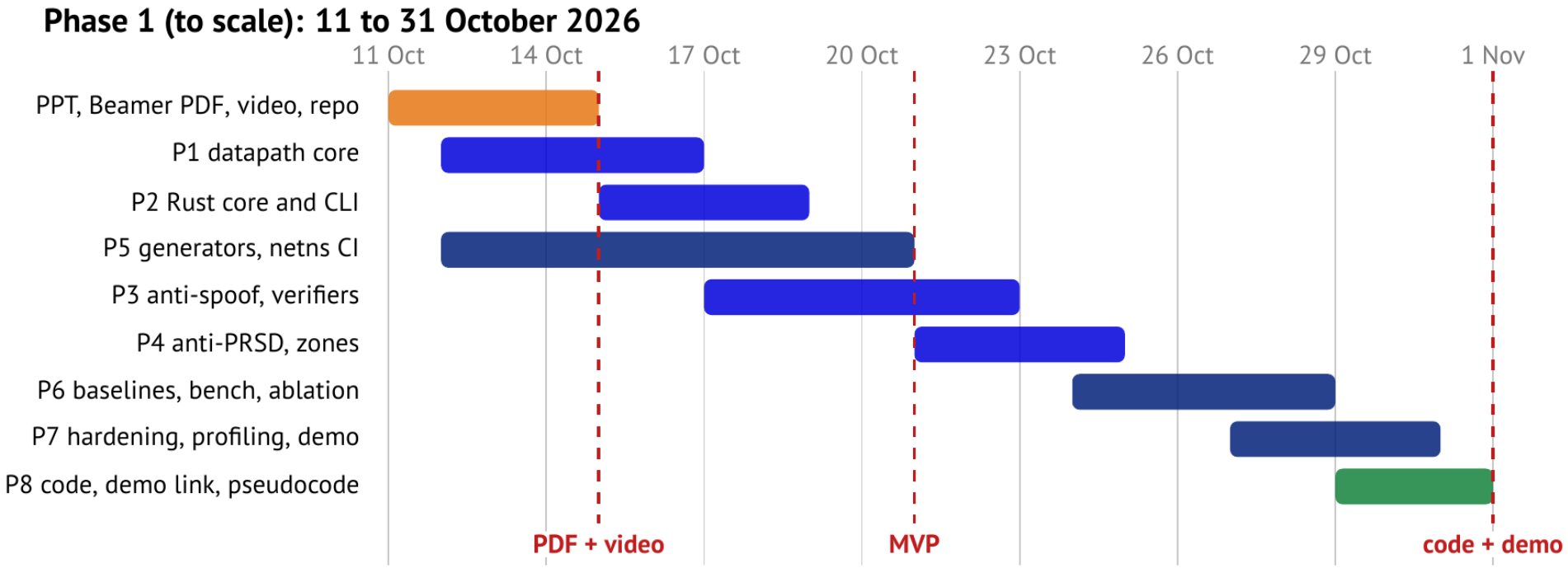
PS-022 requirement	Where it is met
Host pair, controlled lab	Hyper-V target + generator, private switch
Bounded UDP/DNS parsing	Fixed loop limits, verifier-checked
Per-source and global limits	GCRA source table + leased pool
3 synthetic patterns	Scenarios A, B, C (+ D)
No filter and nftables baselines	Same runs, same conditions
XDP-capable NIC or generic mode	hv_netvsc native + generic, both measured
Throughput, CPU, success, drops	Bench metrics
No public flood testing	Generator locked to lab ranges

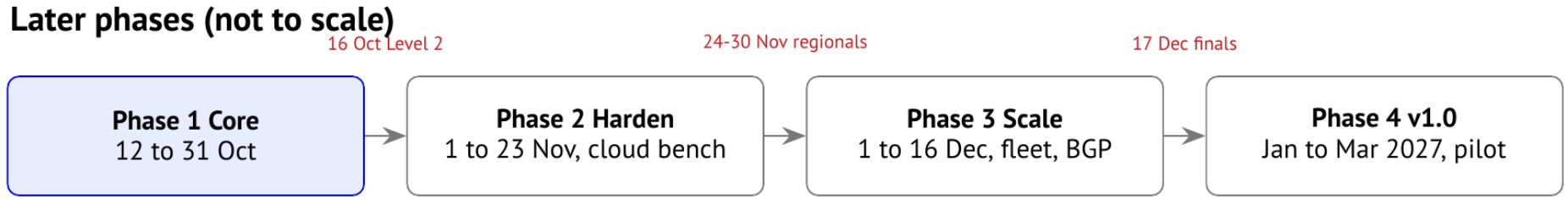
Beyond the Lab: Real-World Deployment

The same program is designed for clouds and ISP fleets; these are roadmap items after the hackathon core.

Need	Design
Cloud VMs	Native XDP on AWS (ENA), Azure (netvsc, MANA), Google Cloud (gVNIC) [15]; segfault doctor applies MTU, LRO and channel settings, falls back to generic mode
Indian mobile users (CGNAT)	Trust keyed by client cookie, so users sharing one address are counted separately
ISP scale	Fleet controller shares heavy hitters and zone flags; shared cookie secret for anycast
Floods larger than the link	Upstream signalling: BGP FlowSpec or RTBH [12], DOTS to scrubbing providers [13]
Production readiness	.deb/.rpm, container, Helm; versioned config and maps; Prometheus; fuzzing; kernel test matrix
Privacy	No full query logs by default; truncated client IPs; aligned with India's DPDP Act 2023

Timeline of Delivery





An MVP that meets every PS-022 requirement lands by 20 Oct; Phase 1 scope is frozen.

Conclusion

Segfault keeps a Linux DNS server answering real users during spoofed, distributed and pulsed floods.

Summary

- Decides in the NIC driver for about 100 to 200 ns per packet
- Spoof-resistant: sketch-first state, two spoof-proof verifiers
- Fair: leased global pool, CGNAT-aware, flash-crowd aware
- Measured honestly against no filter and nftables, with ablation

What we do not claim

- Saving a server whose link is full
- Universal PRSD detection
- Eliminating reflection attacks

Next steps

- Phase 1 code and demo by 31 Oct
- Cloud benchmarks and hardening for the regionals
- Shadow-mode pilot on a real resolver; open-source v1.0

Bibliography I

[1] T. Høiland-Jørgensen et al. “The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel.” ACM CoNEXT (2018).

[2] W. Toorop, L. Hendriks. “XDPeriments: Tinkering with DNS and XDP.” DNS-OARC 34 (2021).

[3] NLnet Labs. “Journeying into XDP: Augmenting the DNS.” APNIC Blog (Oct. 2020).

[4] NLnet Labs. “Journeying into XDP: Fully-fledged DNS service augmentation.” APNIC Blog (Feb. 2022).

[5] G. Cormode, S. Muthukrishnan. “An improved data stream summary: the count-min sketch and its applications.” Journal of Algorithms 55.1 (2005).

[6] S. Miano et al. “Fast In-kernel Traffic Sketching in eBPF.” ACM SIGCOMM Computer Communication Review (2023).

[7] P. Mockapetris. RFC 1035: Domain Names, Implementation and Specification (1987).

[8] J. Dickinson et al. RFC 7766: DNS Transport over TCP, Implementation Requirements (2016).

[9] D. Eastlake 3rd, M. Andrews. RFC 7873: Domain Name System (DNS) Cookies (2016).

Bibliography II

[10] O. Surý, W. Toorop, D. Eastlake 3rd, M. Andrews. RFC 9018: Interoperable Domain Name System (DNS) Server Cookies (2021).

[11] K. Fujiwara, A. Kato, W. Kumari. RFC 8198: Aggressive Use of DNSSEC-Validated Cache (2017).

[12] C. Loibl et al. RFC 8955: Dissemination of Flow Specification Rules (2020).

[13] M. Boucadair, J. Shallow, T. Reddy.K. RFC 9132: DOTS Signal Channel Specification (2021).

[14] NLnet Labs. Unbound 1.18.0 and 1.21.0 release notes: DNS cookies and cookie-secret rollover (2023, 2024).

[15] Linux kernel documentation: BPF, Hyper-V netvsc, Amazon ENA drivers. docs.kernel.org.

[16] P. Vixie, V. Schryver. “DNS Response Rate Limiting (DNS RRL).” ISC Technical Note (2012).

[17] ITU-T. Recommendation I.371: Traffic control and congestion control in B-ISDN (Generic Cell Rate Algorithm).

Thank You for Your Attention!